

CS2610: Computer Organization and Architecture Lab

Lab 0

Harshvardhan Jaju

Roll No: CS24B076

Part 1: Simulating an Assembly Program

Q1. What is the address of main function?

Answer:

The address of the `main` function is:

`0x00000000000010150`

This was obtained from the symbol table using:

```
riscv64-unknown-elf-objdump -t my_RISCV_executable | grep main
```

Q2. How does each instruction affect the register states?

Answer:

- `la t0, num1`
Loads the memory address of variable `num1` into register `t0`.
- `lw t1, 0(t0)`
Loads the 32-bit value stored at address `num1` into register `t1` (`t1 = 15`).
- `la t0, num2`
Loads the memory address of variable `num2` into register `t0`.
- `lw t2, 0(t0)`
Loads the 32-bit value stored at address `num2` into register `t2` (`t2 = 10`).
- `mul t3, t1, t2`
Multiplies the values in registers `t1` and `t2`, storing the result (`150`) in `t3`.

- `la t0, result`
Loads the memory address of variable `result` into register `t0`.
- `sw t3, 0(t0)`
Stores the value in register `t3` into memory at the address `result`. Registers remain unchanged.
- `li a7, 93`
Loads the system call number for program exit (93) into register `a7`.
- `ecall`
Triggers a system call using the value in `a7`, terminating the program.

Q3. What are the addresses of the data variables `num1`, `num2`, and `result`?

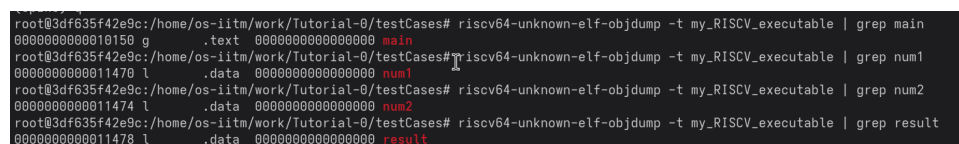
Answer:

Variable	Address
<code>num1</code>	0x00000000000011470
<code>num2</code>	0x00000000000011474
<code>result</code>	0x00000000000011478

These addresses were obtained using:

```
riscv64-unknown-elf-objdump -t my_RISCV_executable | grep num1
riscv64-unknown-elf-objdump -t my_RISCV_executable | grep num2
riscv64-unknown-elf-objdump -t my_RISCV_executable | grep result
```

The variables are placed consecutively in memory, each separated by 4 bytes, which matches the size of a `.word`.



```
root@3df635f42e9c:/home/os-iiit/work/Tutorial-0/testCases# riscv64-unknown-elf-objdump -t my_RISCV_executable | grep main
0000000000010150 g .text 0000000000000000 main
root@3df635f42e9c:/home/os-iiit/work/Tutorial-0/testCases# riscv64-unknown-elf-objdump -t my_RISCV_executable | grep num1
0000000000011470 l .data 0000000000000000 num1
root@3df635f42e9c:/home/os-iiit/work/Tutorial-0/testCases# riscv64-unknown-elf-objdump -t my_RISCV_executable | grep num2
0000000000011474 l .data 0000000000000000 num2
root@3df635f42e9c:/home/os-iiit/work/Tutorial-0/testCases# riscv64-unknown-elf-objdump -t my_RISCV_executable | grep result
0000000000011478 l .data 0000000000000000 result
```

Figure 1: Address

Q4. Create a dump file. Are there differences between the ASM code and the dump? Explain them.

Answer:

The dump file shows the actual machine-level instructions generated after assembling and linking, while the ASM file contains pseudo-instructions and simplified representations for readability.

Thus, the dump represents the exact instructions executed by the processor, while the ASM file is a higher-level, human-friendly version.

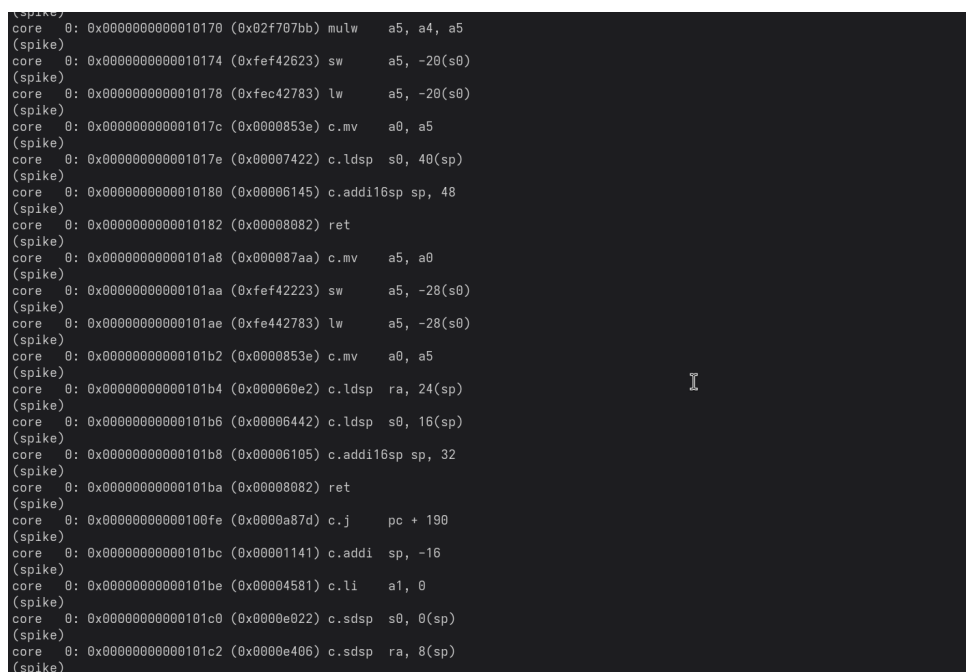
Part 2: Observing Register States with C Code

Q1. Which registers store intermediate results?

Answer:

The multiplication result (200) is first stored in register `a5`, which holds the intermediate result of the `mul` operation. This value is temporarily saved to the stack and later reloaded into `a5`. Before returning from the function, the value is moved into register `a0`, which is used to return the final result according to the RISC-V calling convention.

Register `s0` is only used as a frame pointer and does not store the multiplication result.



```

(spike)
core 0: 0x00000000000010170 (0x02f707bb) mulw   a5, a4, a5
(spike)
core 0: 0x00000000000010174 (0xfef42623) sw     a5, -20(s0)
(spike)
core 0: 0x00000000000010178 (0xfec42783) lw     a5, -20(s0)
(spike)
core 0: 0x0000000000001017c (0x0000853e) c.mv    a0, a5
(spike)
core 0: 0x0000000000001017e (0x00007422) c.ldsp   s0, 40(sp)
(spike)
core 0: 0x00000000000010180 (0x00006145) c.addi16sp sp, 48
(spike)
core 0: 0x00000000000010182 (0x00008082) ret
(spike)
core 0: 0x000000000000101a8 (0x000087aa) c.mv     a5, a0
(spike)
core 0: 0x000000000000101aa (0xfef42223) sw     a5, -28(s0)
(spike)
core 0: 0x000000000000101ae (0xfe442783) lw     a5, -28(s0)
(spike)
core 0: 0x000000000000101b2 (0x0000853e) c.mv     a0, a5
(spike)
core 0: 0x000000000000101b4 (0x000060e2) c.ldsp   ra, 24(sp)
(spike)
core 0: 0x000000000000101b6 (0x00006442) c.ldsp   s0, 16(sp)
(spike)
core 0: 0x000000000000101b8 (0x00006105) c.addi16sp sp, 32
(spike)
core 0: 0x000000000000101ba (0x00008082) ret
(spike)
core 0: 0x000000000000100fe (0x0000a87d) c.j      pc + 190
(spike)
core 0: 0x000000000000101bc (0x00001141) c.addi   sp, -16
(spike)
core 0: 0x000000000000101be (0x00004581) c.li     a1, 0
(spike)
core 0: 0x000000000000101c0 (0x0000e022) c.sdsp   s0, 0(sp)
(spike)
core 0: 0x000000000000101c2 (0x0000e406) c.sdsp   ra, 8(sp)
(spike)

```

Figure 2: Spike simulator instructions

Q2. Which register contains the final result at the end of `main()`?

Answer:

Register `a0` stores the final result of the program.

According to the RISC-V calling convention, function return values are placed in **a0**. After execution, **a0** contains the value:

0xC8 (200 in decimal)

This matches the result of 20×10 , which is returned by the **mul** function and then returned by **main()**.

```
(spike)
core  0: 0x000000000001046c (0x00000073) ecall
core  0: exception trap_user_ecall, epc 0x000000000001046c
(spike) reg 0
zero: 0x0000000000000000  ra: 0x00000000000101da  sp: 0x00000003ffffffb30  gp: 0x0000000000011cb0
tp: 0x0000000000000000  t0: 0x0000000000010270  t1: 0x000000000000000f  t2: 0x0000000000000000
s0: 0x00000000000000c8  s1: 0x0000000000000000  a0: 0x00000000000000c8  a1: 0x0000000000000000
a2: 0x0000000000000000  a3: 0x0000000000000001  a4: 0x0000000000000000  a5: 0x0000000000000000
a6: 0x000000000000001f  a7: 0x000000000000005d  s2: 0x0000000000000000  s3: 0x0000000000000000
s4: 0x0000000000000000  s5: 0x0000000000000000  s6: 0x0000000000000000  s7: 0x0000000000000000
s8: 0x0000000000000000  s9: 0x0000000000000000  s10: 0x0000000000000000  s11: 0x0000000000000000
t3: 0x0000000000000000  t4: 0x0000000000000000  t5: 0x0000000000000000  t6: 0x0000000000000000
(spike)
```

Figure 3: Final Result

Q3. Compare and explain differences between the generated ASM code and the original ASM code from Part 1.

Answer:

The generated assembly code from the C program is more complex than the original hand-written assembly code from Part 1 due to compiler-added function handling and stack management.

- The generated code includes instructions such as stack pointer adjustments (**addi sp, sp, -...**), saving and restoring registers (**sd s0, ...** and **ld s0, ...**), and returning via **jal ra**. These are not present in the original assembly because it was a simple standalone program without separate functions.
- The compiler stores variables and intermediate results on the stack using frame pointer **s0**, whereas the original assembly directly used registers and global memory variables. This results in additional **sw** and **lw** instructions for saving and loading local variables.
- The generated code follows the RISC-V calling convention, passing function arguments in **a0** and **a1** and returning results in **a0**. The original assembly did not explicitly demonstrate a function call structure.
- The compiler-generated assembly may use different registers (such as **a4**, **a5**) and extra move instructions (**mv**) to manage data flow, while the original assembly was more direct.